Central de Notas – Comm

Prestação de Contas TI · Status do projeto e próximos passos

Atualizado em 05/11/2026 · Repositório TecnologiaComm/Central_notas (branch main)

1. Resumo

O app lê a caixa **prestacaocontasti@commcenter.com.br**, separa os anexos (NF, FT, XML, boleto e memória de cálculo), renomeia os arquivos no padrão do time, pré-preenche a linha da planilha, pede o "de acordo" ao Robson e controla PC, chamado e status até a NF ser lançada. O analista passa de **fazer** para **conferir e aprovar**.

Parte	Situação
App (fase 1): caixa, anexos, nomenclatura, planilha, sugestões, alertas, aprovação	**Pronto** · testado em modo demo
Versão Azure: login Microsoft, SharePoint, carga pela web, publicação automática	**Código pronto** · falta criar os recursos e configurar
Código no GitHub	**Na main** · pull request #1 aprovado; testes do GitHub Actions passando
Testes automáticos	13 testes passando (fluxo completo, regras, login, SharePoint, carga)
Ambiente real (M365, Azure, SharePoint)	**Não configurado** · app ainda não está no ar

Importante: nada foi testado ainda com a caixa real, o Azure real ou o SharePoint real. Os testes usam simuladores. Os próximos passos (seção 4) são justamente ligar cada peça ao ambiente da Comm.

2. Como era × como fica

#	Hoje (manual)	No app
1	Verificar o e-mail procurando notas novas	**Automático**: verificação a cada 5 min (ou botão "Verificar e-mails agora").
2	Identificar se é nota, boleto ou memória de cálculo	**Automático**: classifica cada anexo e junta NF, XML, boleto e MC da mesma nota.
3	Salvar e renomear os arquivos	**Automático**: PRESTADOR_NFouFT_NÚMERO_EMPRESA (+ _BOLETO / _MC) em <EMPRESA>/<AAAA-MM>/ e cópia no SharePoint.
4	Preencher a linha da planilha	**Semiautomático**: empresa/filial pelo CNPJ do tomador; prestador e código Protheus pelo CNPJ do prestador; despesa, produto, CC e conta pelo histórico. O analista confere.
5	Enviar e-mail pedindo aprovação	**Um clique**: e-mail ao Robson com anexos e botões "De acordo / Não de acordo" (ou ele responde "De acordo").

#	Hoje (manual)	No app
6	Lançar no Protheus	**Continua manual**: depois do “de acordo”, o analista lança no Protheus e registra no app PC, chamado, status e data. Com “NF LANÇADA”, a nota sai da fila.

Fluxo de etapas da nota

```
Em conferência → Aguardando aprovação → Aprovada / Reprovada → NF lançada (ou Ignorada, quando não é nota)
```

Alertas automáticos

- **Duplicidade**: mesma chave de acesso, ou mesma NFS do mesmo prestador no app ou na planilha antiga.
- **Fornecedor**: CNPJ sem cadastro ou sem código Protheus para aquela empresa.
- **Empresa/filial** não identificada pelo CNPJ do tomador.
- **Prazos e valores**: vencimento em até 3 dias, boleto com valor diferente da nota, campos não lidos.

3. O que já foi feito

3.1 Aplicativo (fase 1)

- Leitura da caixa pelo **Microsoft Graph** (permissões de aplicativo, restritas à caixa da prestação de contas).
- Leitura dos dados: **XML** de NF-e/NFS-e › **PDF** (texto e linha digitável do boleto) › **IA opcional** para PDFs sem XML.
- Nomenclatura e pastas no padrão do time; agrupamento de anexos que chegam em e-mails diferentes.
- Sugestões a partir do histórico (cerca de 730 lançamentos de jul/25 a nov/26 da planilha atual).
- Pedido de “de acordo” com links assinados (o Safe Links do Outlook não aprova sozinho) e leitura das respostas por e-mail.
- **Planilha** com uma aba por mês nas mesmas 17 colunas de hoje + aba “Controle app”; cópia sempre atualizada.

3.2 Versão Azure

- **Login com a conta Microsoft da Comm**: o histórico registra o e-mail real de quem conferiu e aprovou; lista opcional de usuários permitidos.
- **SharePoint**: arquivos renomeados e a planilha CONTROLE_NOTAS.xlsx copiados para a biblioteca; reenvio automático se falhar.
- **Dados persistentes** em /home/data (banco, arquivos e carga), que não se perdem nas publicações.
- Tela **“Carga da planilha”**: importa a planilha atual pelo navegador.
- **Publicação automática**: GitHub Actions roda os testes e publica no App Service a cada alteração na main.
- Script **azure/provisionar.sh** cria tudo no Azure com um comando; passo a passo completo no README.

Arquitetura

```
GitHub (main) ⟶ GitHub Actions: testes ⟶ Azure App Service (B1 Linux, Always On)
  ├─ login Microsoft (Entra ID)
  ├─ Microsoft Graph: caixa prestacaocontasti@ (ler e enviar)
  ├─ Microsoft Graph: SharePoint (arquivos e planilha)
  └─ /home/data: banco SQLite + arquivos
```

4. O que precisamos fazer

Em ordem. Cada bloco indica quem faz. Os detalhes técnicos (comandos e telas) estão no README do repositório, seção “Publicar no Azure”.

Passo 1 – Decidir os nomes e valores · Renan / time administrativo

- ☐ Nome do site no Azure (ex.: **central-notas-comm** → https://central-notas-comm.azurewebsites.net).
- ☐ Site e biblioteca do SharePoint para as notas (ex.: “Prestação de Contas TI” / “Documentos” / pasta “Notas”).
- ☐ Quem pode entrar no app (lista de e-mails ou grupo) e se a regra de aprovação continua só o Robson.
- ☐ Juntar a branch na main — **feito** (pull request #1).

Passo 2 – Registro do app no Microsoft 365 · Admin do M365 / TI

- ☐ Criar o registro “Central de Notas” (somente este diretório) e anotar **Tenant ID** e **Client ID**.
- ☐ Permissões de aplicativo do Graph: **Mail.ReadWrite**, **Mail.Send**, **Sites.Selected** + consentimento do administrador.
- ☐ Criar o **segredo do cliente** (anotar o valor e a data de expiração).
- ☐ Autenticação Web: URI **https://<app>.azurewebsites.net/.auth/login/aad/callback** e “Tokens de ID” marcado.
- ☐ PowerShell: restringir o app à caixa prestacaocontasti@ (ApplicationAccessPolicy).
- ☐ PowerShell: dar escrita só no site das notas (New-MgSitePermission).
- ☐ Opcional: “Atribuição necessária = Sim” e adicionar o time e o Robson ao app.

Passo 3 – Criar os recursos no Azure · TI (com acesso à assinatura Azure)

- ☐ Confirmar se a Comm já tem assinatura do Azure (senão, criar com o admin do M365).
- ☐ No Azure Cloud Shell: **TENANT_ID=… CLIENT_ID=… APP=central-notas-comm bash azure/provisionar.sh**
- ☐ Em Configurações do App Service: **GRAPH_CLIENT_SECRET**, **MICROSOFT_PROVIDER_AUTHENTICATION_SECRET**, **SHAREPOINT_SITE** e, se quiser, **USUARIOS_PERMITIDOS** e **ANTHROPIC_API_KEY**.
- ☐ Conferir o valor mensal do plano B1 na página de preços do App Service.

Passo 4 – Ligar o GitHub ao Azure · Quem administra o GitHub da TecnologiaComm

- ☐ Baixar o perfil de publicação do App Service.
- ☐ No GitHub (Settings → Secrets and variables → Actions): segredo **AZURE_WEBAPP_PUBLISH_PROFILE** e variável **AZURE_WEBAPP_NAME**.
- ☐ Rodar de novo o workflow (aba Actions → “Testar e publicar no Azure” → Run workflow) e conferir que a publicação ficou verde.

Passo 5 – Primeiro uso · Time administrativo

- ☐ Entrar no site com a conta da Comm.
- ☐ Menu “Carga da planilha”: enviar a planilha atual (aba Infos + abas mensais).
- ☐ “Verificar e-mails agora” e conferir as primeiras notas (empresa, filial, fornecedor, CC, conta).
- ☐ Rodar **alguns dias só conferindo**, sem pedir “de acordo”, e anotar o que precisar ajustar.
- ☐ Depois, liberar o envio do “de acordo” ao Robson.

5. Pontos de atenção

Ponto	O que fazer
Valores de exemplo	Nome do site SharePoint e da biblioteca são exemplos: confirmar antes de usar.
Uma instância só	Não ligar escala horizontal no App Service: a verificação da caixa roda dentro do app.
Banco	SQLite em /home/data atende o volume atual. Se crescer, migrar para Azure SQL ou PostgreSQL (DATABASE_URL).
Segredo do cliente	Tem validade: anotar a data e renovar antes de vencer (senão o app para de ler a caixa).
Anexos grandes	O envio de e-mail pelo Graph aceita anexos de até ~3 MB cada.
Backup	Ativar Backups do App Service (cobre /home/data).

6. Cronograma sugerido

Quando	O quê	Quem
Semana 1	Passos 1 a 4: decisões, registro no M365, Azure e GitHub	TI + Renan
Semana 2	Passo 5: carga da planilha e alguns dias só conferindo a caixa real	Time administrativo
Semana 3	Ajustes do que apareceu na conferência; liberar o “de acordo” ao Robson e usar no dia a dia	Renan + Robson